K_vecinos_más_cercanos(RH)

October 27, 2026

1 Universidad Autónoma de Zacatecas

1.0.1 Licenciatura en Actuaría

2 Modelo de clasificación KNN (K vecinos más cercanos)

2.0.1 Predicción del abandono de empleados en Recursos Humanos

Alumna: Ana Camila López Ceballos.

[8]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import os
```

[9]:
```
data = pd.read_csv("recursos_humanos (3).csv")
data
```

[9]:

	satisfaction_level	last_evaluation	number_project	\
0	0.38	0.53	2	
1	0.80	0.86	5	
2	0.11	0.88	7	
3	0.72	0.87	5	
4	0.37	0.52	2	
…	…	…	…	
14994	0.40	0.57	2	
14995	0.37	0.48	2	
14996	0.37	0.53	2	
14997	0.11	0.96	6	
14998	0.37	0.52	2	

	average_montly_hours	time_spend_company	Work_accident	left	\
0	157	3	0	1	
1	262	6	0	1	
2	272	4	0	1	
3	223	5	0	1	
4	159	3	0	1	
…	…	…	…	…	

```
14994                   151                      3              0     1
14995                   160                      3              0     1
14996                   143                      3              0     1
14997                   280                      4              0     1
14998                   158                      3              0     1

       promotion_last_5years    sales  salary
0                          0    sales     low
1                          0    sales  medium
2                          0    sales  medium
3                          0    sales     low
4                          0    sales     low
…                        …      …      …
14994                      0  support     low
14995                      0  support     low
14996                      0  support     low
14997                      0  support     low
14998                      0  support     low

[14999 rows x 10 columns]
```

[10]:
```
data.head
```

[10]:
```
<bound method NDFrame.head of        satisfaction_level  last_evaluation
number_project  \
0                    0.38            0.53               2
1                    0.80            0.86               5
2                    0.11            0.88               7
3                    0.72            0.87               5
4                    0.37            0.52               2
…                    …               …                  …
14994                0.40            0.57               2
14995                0.37            0.48               2
14996                0.37            0.53               2
14997                0.11            0.96               6
14998                0.37            0.52               2

       average_montly_hours  time_spend_company  Work_accident  left  \
0                       157                   3              0     1
1                       262                   6              0     1
2                       272                   4              0     1
3                       223                   5              0     1
4                       159                   3              0     1
…                       …                     …              …     …
14994                   151                   3              0     1
14995                   160                   3              0     1
14996                   143                   3              0     1
```

```
14997                  280                     4              0     1
14998                  158                     3              0     1

       promotion_last_5years    sales  salary
0                          0    sales     low
1                          0    sales  medium
2                          0    sales  medium
3                          0    sales     low
4                          0    sales     low
…                        …      …      …
14994                      0  support     low
14995                      0  support     low
14996                      0  support     low
14997                      0  support     low
14998                      0  support     low

[14999 rows x 10 columns]>
```

Observamos los primeros registros para conocer las variables de nuestra base de datos y la información que contiene cada una.

[11]:
```
data = pd.get_dummies(data, columns=["sales", "salary"], dtype=int)
data
```

[11]:
```
       satisfaction_level  last_evaluation  number_project  \
0                    0.38             0.53               2
1                    0.80             0.86               5
2                    0.11             0.88               7
3                    0.72             0.87               5
4                    0.37             0.52               2
…                    …                …                  …
14994                0.40             0.57               2
14995                0.37             0.48               2
14996                0.37             0.53               2
14997                0.11             0.96               6
14998                0.37             0.52               2

       average_montly_hours  time_spend_company  Work_accident  left  \
0                       157                   3              0     1
1                       262                   6              0     1
2                       272                   4              0     1
3                       223                   5              0     1
4                       159                   3              0     1
…                       …                     …              …     …
14994                   151                   3              0     1
14995                   160                   3              0     1
14996                   143                   3              0     1
```

```
14997                  280                  4             0     1
14998                  158                  3             0     1

       promotion_last_5years  sales_IT  sales_RandD  …  sales_hr  \
0                          0         0            0  …         0
1                          0         0            0  …         0
2                          0         0            0  …         0
3                          0         0            0  …         0
4                          0         0            0  …         0
…                        …       …          …   …       …
14994                      0         0            0  …         0
14995                      0         0            0  …         0
14996                      0         0            0  …         0
14997                      0         0            0  …         0
14998                      0         0            0  …         0

       sales_management  sales_marketing  sales_product_mng  sales_sales  \
0                     0                0                  0            1
1                     0                0                  0            1
2                     0                0                  0            1
3                     0                0                  0            1
4                     0                0                  0            1
…                   …                …                  …          …
14994                 0                0                  0            0
14995                 0                0                  0            0
14996                 0                0                  0            0
14997                 0                0                  0            0
14998                 0                0                  0            0

       sales_support  sales_technical  salary_high  salary_low  salary_medium
0                  0                0            0           1              0
1                  0                0            0           0              1
2                  0                0            0           0              1
3                  0                0            0           1              0
4                  0                0            0           1              0
…                …                …          …         …            …
14994              1                0            0           1              0
14995              1                0            0           1              0
14996              1                0            0           1              0
14997              1                0            0           1              0
14998              1                0            0           1              0

[14999 rows x 21 columns]
```

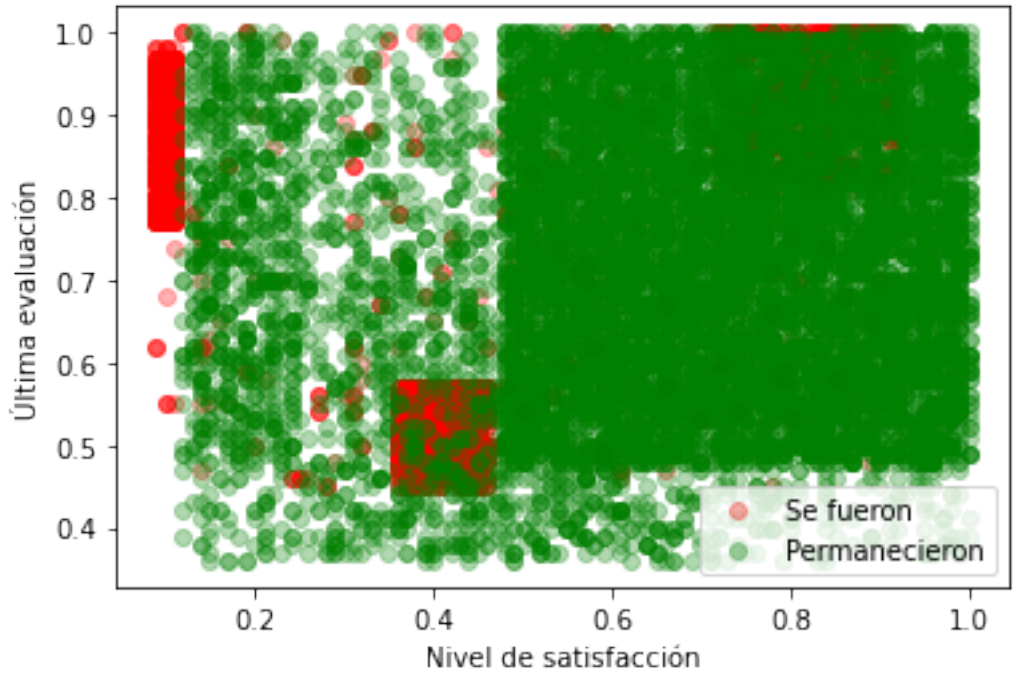
Convertimos las variables categóricas `sales` y `salary` en variables numéricas mediante `pd.get_dummies()`.

[12]:
```
# Empleados que dejaron y que permanecieron en la empresa
S = data[data.left == 1]
N = data[data.left == 0]
```

Dividimos a los empleados según la variable `left`:

- **1:** Empleados que abandonaron la empresa.
- **0:** Empleados que permanecieron.

[13]:
```
# Diagrama de dispersión
plt.scatter(S.satisfaction_level, S.last_evaluation, color="red", label="Se␣
 ↪fueron", alpha=0.3)
plt.scatter(N.satisfaction_level, N.last_evaluation, color="green",␣
 ↪label="Permanecieron", alpha=0.3)
plt.xlabel("Nivel de satisfacción")
plt.ylabel("Última evaluación")
plt.legend()
plt.show()
```



Elaboramos un diagrama de dispersión para observar la relación entre el nivel de satisfacción y la última evaluación de los empleados.

Los puntos rojos representan a quienes abandonaron la empresa y los verdes a quienes permanecieron.

[14]:
```
# Separación de variables
y = data.left.values
x_data = data.drop(["left"], axis=1)
y
```

[14]:
```
array([1, 1, 1, …, 1, 1, 1])
```

Definimos `y` como nuestra variable dependiente (`left`), que indica si el empleado abandonó la empresa.

En `x_data` colocamos las variables independientes que utilizaremos para realizar las predicciones.

[15]:
```
# Normalización de los datos
x = (x_data - np.min(x_data))/(np.max(x_data) - np.min(x_data))
x
```

```
/opt/conda/envs/anaconda-2022.05-py39/lib/python3.9/site-
packages/numpy/core/fromnumeric.py:84: FutureWarning: In a future version,
DataFrame.min(axis=None) will return a scalar min over the entire DataFrame. To
retain the old behavior, use 'frame.min(axis=0)' or just 'frame.min()'
  return reduction(axis=axis, out=out, **passkwargs)
/opt/conda/envs/anaconda-2022.05-py39/lib/python3.9/site-
packages/numpy/core/fromnumeric.py:84: FutureWarning: In a future version,
DataFrame.max(axis=None) will return a scalar max over the entire DataFrame. To
retain the old behavior, use 'frame.max(axis=0)' or just 'frame.max()'
  return reduction(axis=axis, out=out, **passkwargs)
```

[15]:

	satisfaction_level	last_evaluation	number_project	\
0	0.318681	0.265625	0.0	
1	0.780220	0.781250	0.6	
2	0.021978	0.812500	1.0	
3	0.692308	0.796875	0.6	
4	0.307692	0.250000	0.0	
…	…	…	…	
14994	0.340659	0.328125	0.0	
14995	0.307692	0.187500	0.0	
14996	0.307692	0.265625	0.0	
14997	0.021978	0.937500	0.8	
14998	0.307692	0.250000	0.0	

	average_montly_hours	time_spend_company	Work_accident	\
0	0.285047	0.125	0.0	
1	0.775701	0.500	0.0	
2	0.822430	0.250	0.0	
3	0.593458	0.375	0.0	
4	0.294393	0.125	0.0	
…	…	…	…	
14994	0.257009	0.125	0.0	

```
14995              0.299065               0.125           0.0
14996              0.219626               0.125           0.0
14997              0.859813               0.250           0.0
14998              0.289720               0.125           0.0

       promotion_last_5years  sales_IT  sales_RandD  sales_accounting  \
0                        0.0       0.0          0.0               0.0
1                        0.0       0.0          0.0               0.0
2                        0.0       0.0          0.0               0.0
3                        0.0       0.0          0.0               0.0
4                        0.0       0.0          0.0               0.0
…                        …         …            …                 …
14994                    0.0       0.0          0.0               0.0
14995                    0.0       0.0          0.0               0.0
14996                    0.0       0.0          0.0               0.0
14997                    0.0       0.0          0.0               0.0
14998                    0.0       0.0          0.0               0.0

       sales_hr  sales_management  sales_marketing  sales_product_mng  \
0           0.0               0.0              0.0                0.0
1           0.0               0.0              0.0                0.0
2           0.0               0.0              0.0                0.0
3           0.0               0.0              0.0                0.0
4           0.0               0.0              0.0                0.0
…           …                 …                …                  …
14994       0.0               0.0              0.0                0.0
14995       0.0               0.0              0.0                0.0
14996       0.0               0.0              0.0                0.0
14997       0.0               0.0              0.0                0.0
14998       0.0               0.0              0.0                0.0

       sales_sales  sales_support  sales_technical  salary_high  salary_low  \
0              1.0            0.0              0.0          0.0         1.0
1              1.0            0.0              0.0          0.0         0.0
2              1.0            0.0              0.0          0.0         0.0
3              1.0            0.0              0.0          0.0         1.0
4              1.0            0.0              0.0          0.0         1.0
…              …              …                …            …           …
14994          0.0            1.0              0.0          0.0         1.0
14995          0.0            1.0              0.0          0.0         1.0
14996          0.0            1.0              0.0          0.0         1.0
14997          0.0            1.0              0.0          0.0         1.0
14998          0.0            1.0              0.0          0.0         1.0

       salary_medium
0                0.0
1                1.0
```

```
2                1.0
3                0.0
4                0.0
…                …
14994            0.0
14995            0.0
14996            0.0
14997            0.0
14998            0.0

[14999 rows x 20 columns]
```

Normalizamos los datos para que las variables tengan valores entre 0 y 1.

Esto evita que las variables con valores más grandes tengan demasiada influencia al calcular las distancias entre empleados.

[16]:
```
# Bases de entrenamiento y prueba
from sklearn.model_selection import train_test_split
x_train, x_test, y_train, y_test = train_test_split(x, y, test_size=0.30,␣
 ↪random_state=1)
```

Dividimos nuestra base en 70% para entrenamiento y 30% para prueba.

Utilizamos los datos de entrenamiento para construir el modelo y los de prueba para evaluar qué tan bien clasifica a los empleados.

[17]:
```
# Modelo KNN (K vecinos más cercanos)
from sklearn.neighbors import KNeighborsClassifier
knn = KNeighborsClassifier(n_neighbors=3)
knn.fit(x_train, y_train)
prediction = knn.predict(x_test)
print("Score: ", knn.score(x_test, y_test))
```

```
Score:  0.9453333333333334
```

Construimos nuestro primer modelo utilizando los tres vecinos más cercanos.

Entrenamos el modelo y calculamos su Score, que representa la proporción de empleados clasificados correctamente.

[18]:
```
# Comparación de los valores de K
for k in range(1, 21):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(x_train, y_train)
    prediction = knn.predict(x_test)
    print("K =", k, "Score: ", knn.score(x_test, y_test))
```

```
K = 1 Score:  0.9617777777777777
K = 2 Score:  0.9577777777777777
```

```
K = 3 Score:  0.9453333333333334
K = 4 Score:  0.946
K = 5 Score:  0.9393333333333334
K = 6 Score:  0.9446666666666667
K = 7 Score:  0.9411111111111111
K = 8 Score:  0.9393333333333334
K = 9 Score:  0.9324444444444444
K = 10 Score:  0.934
K = 11 Score:  0.9293333333333333
K = 12 Score:  0.9311111111111111
K = 13 Score:  0.9262222222222222
K = 14 Score:  0.9288888888888889
K = 15 Score:  0.9253333333333333
K = 16 Score:  0.9264444444444444
K = 17 Score:  0.9228888888888889
K = 18 Score:  0.9211111111111111
K = 19 Score:  0.9186666666666666
K = 20 Score:  0.9188888888888889
```

Probamos los valores de K desde 1 hasta 20 para comparar la precisión global de cada modelo.

Encontramos que **K = 1 obtuvo el mayor Score, con 96.18%**, por lo que inicialmente seleccionamos este valor.

[22]:
```
# Modelo KNN con K = 1
knn = KNeighborsClassifier(n_neighbors=1)
knn.fit(x_train, y_train)

prediction = knn.predict(x_test)

print("Score: ", knn.score(x_test, y_test))
```

```
Score:  0.9617777777777777
```

[23]:
```
# Modelo KNN con K = 1
knn = KNeighborsClassifier(n_neighbors=1)
knn.fit(x_train, y_train)

prediction = knn.predict(x_test)
```

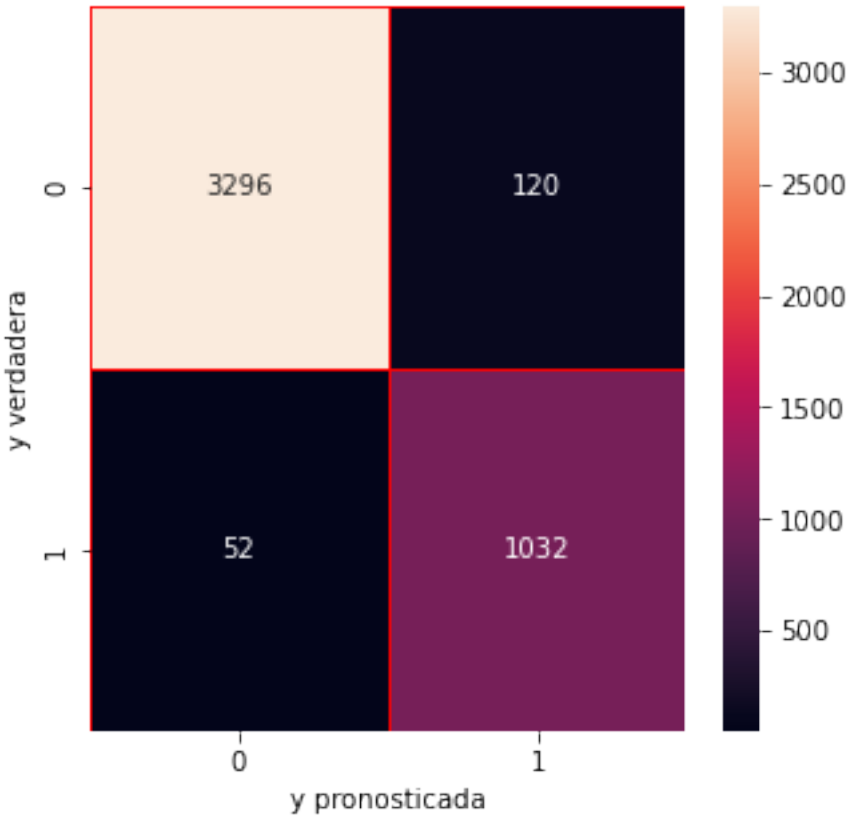
Utilizamos el modelo con K = 1 para obtener las predicciones que compararemos con los resultados reales de los empleados.

[24]:
```
# Creación de la matriz de confusión
from sklearn.metrics import confusion_matrix

cm = confusion_matrix(y_test, prediction)
cm
```

[24]:
```
array([[3296,  120],
       [  52, 1032]])
```

[25]:
```
# Visualización de la matriz de confusión
import seaborn as sns
f, ax = plt.subplots(figsize=(5,5))
sns.heatmap(cm, annot=True, linewidths=0.5, linecolor="red", fmt=".0f", ax=ax)
plt.xlabel("y pronosticada")
plt.ylabel("y verdadera")
plt.show()
```



2.0.2 Interpretación de la matriz de confusión

Obtuvimos los siguientes resultados:

- **3,296 verdaderos negativos:** Empleados que permanecieron y fueron clasificados correctamente.
- **1,032 verdaderos positivos:** Empleados que se fueron y fueron identificados correctamente.
- **120 falsos positivos:** Empleados que permanecieron, pero el modelo predijo que se irían.
- **52 falsos negativos:** Empleados que se fueron, pero el modelo predijo que permanecerían.

En total, clasificamos correctamente a **4,328 de los 4,500 empleados**, obteniendo una precisión

global del 96.18%.

[26]:
```
# Cálculo de las precisiones
correctos = cm[0,0] + cm[1,1]
Incorrectos = cm[1,0] + cm[0,1]

PrecisionGlobal = correctos / (correctos + Incorrectos)
PrecisionNo = cm[0,0] / (cm[0,0] + cm[1,0])
PrecisionSi = cm[1,1] / (cm[1,1] + cm[0,1])

print("Precisión global:", PrecisionGlobal)
print("Precisión de empleados que permanecen:", PrecisionNo)
print("Precisión de empleados que se van:", PrecisionSi)
```

```
Precisión global: 0.9617777777777777
Precisión de empleados que permanecen: 0.984468339307049
Precisión de empleados que se van: 0.8958333333333334
```

Calculamos tres medidas para evaluar nuestro modelo:

- **Precisión global:** 96.18%.
- **Precisión No:** 98.45%.
- **Precisión Sí:** 89.58%.

Esto significa que el modelo tiene una buena precisión general, aunque es más preciso cuando predice que un empleado permanecerá en la empresa.

[27]:
```
# Tabla comparativa de precisiones
resultados = []

for k in range(1, 21):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(x_train, y_train)
    prediction = knn.predict(x_test)

    cm = confusion_matrix(y_test, prediction)

    correctos = cm[0,0] + cm[1,1]
    Incorrectos = cm[1,0] + cm[0,1]

    PrecisionGlobal = correctos / (correctos + Incorrectos)
    PrecisionNo = cm[0,0] / (cm[0,0] + cm[1,0])
    PrecisionSi = cm[1,1] / (cm[1,1] + cm[0,1])

    resultados.append([k, PrecisionGlobal, PrecisionNo, PrecisionSi])

tabla = pd.DataFrame(resultados, columns=["K", "Precisión global", "Precisión␣
 ↪No", "Precisión Sí"])
tabla
```

[27]:

	K	Precisión global	Precisión No	Precisión Sí
0	1	0.961778	0.984468	0.895833
1	2	0.957778	0.968895	0.921698
2	3	0.945333	0.975120	0.859966
3	4	0.946000	0.960790	0.897824
4	5	0.939333	0.962070	0.868972
5	6	0.944667	0.959652	0.895735
6	7	0.941111	0.960269	0.880223
7	8	0.939333	0.955375	0.886559
8	9	0.932444	0.956038	0.858456
9	10	0.934000	0.953475	0.870877
10	11	0.929333	0.953986	0.852022
11	12	0.931111	0.953828	0.858998
12	13	0.926222	0.953796	0.841198
13	14	0.928889	0.953693	0.851103
14	15	0.925333	0.953742	0.838156
15	16	0.926444	0.949577	0.852198
16	17	0.922889	0.950132	0.837764
17	18	0.921111	0.945042	0.843544
18	19	0.918667	0.944866	0.834888
19	20	0.918889	0.943330	0.839471

Elaboramos una tabla con las tres precisiones para cada valor de K, desde 1 hasta 20.

Observamos que K = 1 obtuvo la mayor precisión global (96.18%), mientras que K = 2 obtuvo una mayor precisión al predecir qué empleados abandonarían la empresa (92.17%).

Por eso, también comparamos la sensibilidad de ambos modelos.

[28]:
```
# Comparación de sensibilidad para K = 1 y K = 2
for k in range(1, 3):
    knn = KNeighborsClassifier(n_neighbors=k)
    knn.fit(x_train, y_train)
    prediction = knn.predict(x_test)

    cm = confusion_matrix(y_test, prediction)

    sensibilidad = cm[1,1] / (cm[1,1] + cm[1,0])

    print("K =", k, "Sensibilidad:", sensibilidad)
```

```
K = 1 Sensibilidad: 0.9520295202952029
K = 2 Sensibilidad: 0.9012915129151291
```

Obtuvimos:

- **K = 1:** 95.20%.
- **K = 2:** 90.13%.

Elegimos **K = 1** porque identificó a una mayor proporción de empleados que realmente abandonaron

la empresa y también obtuvo la mayor precisión global.

Esto es importante porque buscamos anticipar posibles salidas de empleados.

[31]:
```
# Modelo KNN con K = 1
knn = KNeighborsClassifier(n_neighbors=1)
knn.fit(x_train, y_train)

prediction = knn.predict(x_test)
```

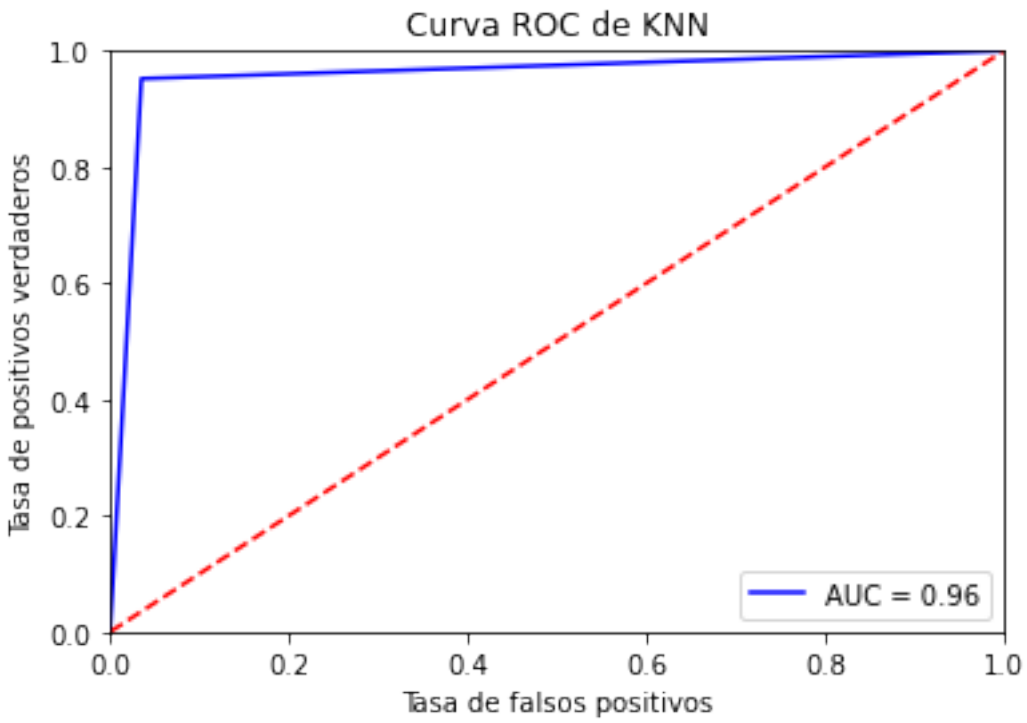
Utilizamos nuevamente K = 1 para evaluar su capacidad de distinguir entre los empleados que abandonan la empresa y los que permanecen mediante la curva ROC.

[32]:
```
from sklearn.metrics import roc_curve
from sklearn.metrics import auc

y_scores = knn.predict_proba(x_test)

fpr, tpr, threshold = roc_curve(y_test, y_scores[:,1])
roc_auc = auc(fpr, tpr)

plt.title("Receiver Operating Characteristic")
plt.plot(fpr, tpr, 'b', label='AUC = %0.2f' % roc_auc)
plt.legend(loc="lower right")
plt.plot([0,1],[0,1], 'r--')
plt.xlim([0,1])
plt.ylim([0,1])
plt.ylabel("Tasa de positivos verdaderos")
plt.xlabel("Tasa de falsos positivos")
plt.title("Curva ROC de KNN")
plt.show()
```



Elaboramos la curva ROC para evaluar la capacidad del modelo de distinguir entre los empleados que permanecen y los que abandonan la empresa.

Obtuvimos un **AUC de 0.96**, lo que indica una muy buena capacidad de discriminación en los datos evaluados.

La curva azul se encuentra cerca de la esquina superior izquierda y por encima de la línea roja, lo que refleja un buen desempeño del modelo.

2.0.3 Conclusiones

Después de evaluar los valores de K desde 1 hasta 20, encontramos que **K = 1 obtuvo la mayor precisión global, con 96.18%**.

En la matriz de confusión obtuvimos 4,328 predicciones correctas y 172 incorrectas. Además, el modelo alcanzó una precisión del 98.45% para los empleados que permanecieron y del 89.58% para aquellos que abandonaron la empresa.

También obtuvimos una sensibilidad del 95.20%, lo que significa que el modelo identificó correctamente a la mayoría de los empleados que realmente abandonaron la empresa.

Finalmente, la curva ROC presentó un **AUC de 0.96**, lo que indica una muy buena capacidad para distinguir entre ambas categorías en los datos evaluados.

[ ]: